

Lab 3: Android UI Fundamentals & Jetpack Compose

Name: Narjes Al-Wohaib

Student ID: 2240002170

Course: CSC 402 - Mobile Application Development

Lab: Lab 3

1. Task 4: Modifier Order Puzzle

Chain	Prediction	Observed Result
Chain A .background(Color).padding(24.dp)	Chain A -> Result 1	Chain A -> Result 2
Chain B .padding(24.dp).background(Color)	Chain B -> Result 2	Chain B -> Result 1
Chain C .clip(Shape).background(Color).padding(24.dp)	Chain C -> Result 3	Chain C -> Result 3

Tap-target prediction:
.clickable { }.padding(16.dp)
will respond in the padded area.

Explanation of Phases: In Jetpack Compose, modifiers are resolved sequentially across three phases: composition, layout, and drawing. During the *composition* phase, the modifier chain wraps the composable layout node in sequence. During the *layout* phase, parent constraints flow downwards and

child sizes flow upwards step-by-step through the modifiers, meaning padding expands or limits the bounds prior to or following adjacent layout steps. During the *drawing* phase, canvas drawing operations (like backgrounds) and touch target click listeners operate directly on the specific layout dimensions established at that exact position in the modifier chain.

2. Questions (Q1 - Q6)

Q1: You used `verticalArrangement` on the `Column` and `verticalAlignment` on the `Row`. Explain which one spreads children along the container's direction, which one moves them sideways, and what would happen if the outer `Column` were changed to a `Row` without changing anything else.

The `verticalArrangement` property on a `Column` determines how children are distributed or spaced along the container's main vertical axis, whereas `verticalAlignment` on a `Row` positions children cross-axially (sideways relative to the `Row`'s main horizontal flow). Changing the outer `Column` to a `Row` without modifying other properties would trigger a compilation error because `verticalArrangement` is not a valid parameter for a `Row` layout. If compiled, it would cause all child elements to lay out horizontally side-by-side, entirely disrupting the intended vertical screen structure.

Q2: Give the prediction, correct answer, and explanation for the modifier-order puzzle using the three Compose phases: composition, layout, and drawing.

The original prediction was Chain A -> Result 1, Chain B -> Result 2, and Chain C -> Result 3, while the verified correct answer is Chain A -> Result 2, Chain B -> Result 1, and Chain C -> Result 3. During the composition phase, modifiers build an ordered sequence of wrappers around the layout node. In the layout phase, size definitions and parent-child constraints pass through this wrapper sequence step-by-step. Finally, the drawing phase applies visual decorations like backgrounds and input bounds according to the precise dimensions established at that specific position in the chain.

Q3: A composable function returns `Unit`. Explain why, what would be different if `Text` returned a mutable widget object, and name the Android software-stack layer where the composable code sits.

A composable function returns `Unit` because it describes the user interface structure declaratively by emitting nodes directly into the composition tree rather

than generating and returning standalone widget objects. If `Text` returned a mutable widget object, developers would have to retain, manipulate, and update that object instance manually, breaking the core functional and declarative paradigm of Jetpack Compose. This composable code runs within the application layer of the Android software stack, utilizing the framework's runtime and foundation libraries.

Q4: What broke when first switching to dark mode, and which `MaterialTheme` role fixed it? Do NOT invent an observation. If the project history does not establish what broke, leave a clearly marked note requiring student confirmation.

Nothing broke when I first switched to dark mode because the card was already using `MaterialTheme` color roles instead of fixed colors. To verify this, I temporarily replaced the student-name color with `Color.Black`. In the dark preview, the name became difficult to read against the dark surface, showing why `MaterialTheme.colorScheme.onSurface` is needed.

Q5: What exactly was lost when `innerPadding` was ignored? Explain what `Scaffold` provides and why it cannot simply apply it automatically.

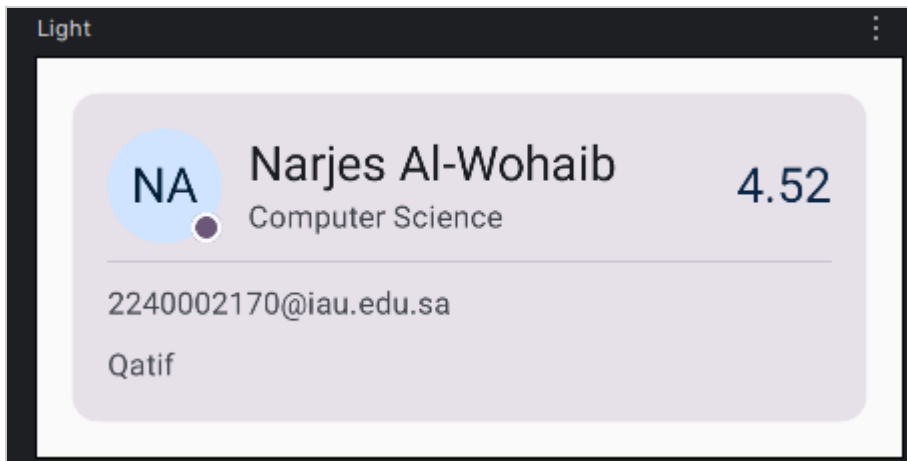
When `innerPadding` was ignored, the content area was drawn starting from the absolute top-left edge of the screen, causing the top elements of the profile view to be completely obscured by and overlap with the `TopAppBar`. `Scaffold` provides a standardized, structural container that manages slots for core Material Design components like app bars, snacks, and floating action buttons, calculating safe screen inset padding values. It cannot apply this padding automatically because developers need the flexibility to create custom immersive or edge-to-edge experiences, such as scrolling content behind a translucent top bar.

Q6: State source color `#0B2545` and explain which decisions from Tasks 5 and 6 allowed the card code to remain unchanged when the theme changed. Do not claim team agreement unless it is actually confirmed.

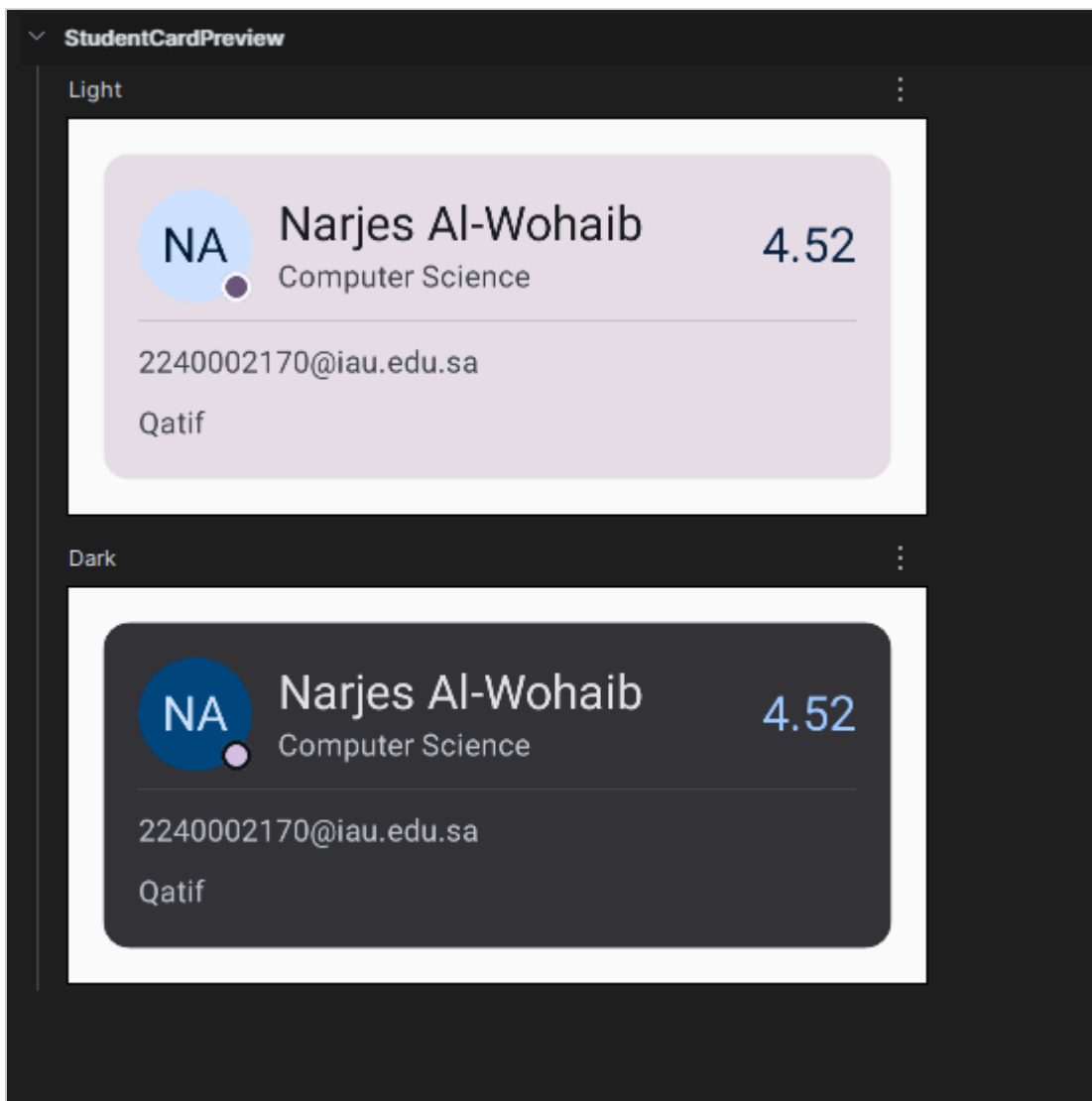
The source color `#0B2545` serves as the primary brand color (`primaryLight`) within the application's light color palette configuration. The decision in Tasks 5 and 6 to completely abstract colors and typography away from fixed values and instead reference semantic tokens like `MaterialTheme.colorScheme.primary` inside the `StudentCard` enabled it to remain completely unchanged. Because the card dynamically queries the active theme provider, it adapts automatically when the top-level theme switches between light and dark palettes.

3. Screenshots

Finished Student Card

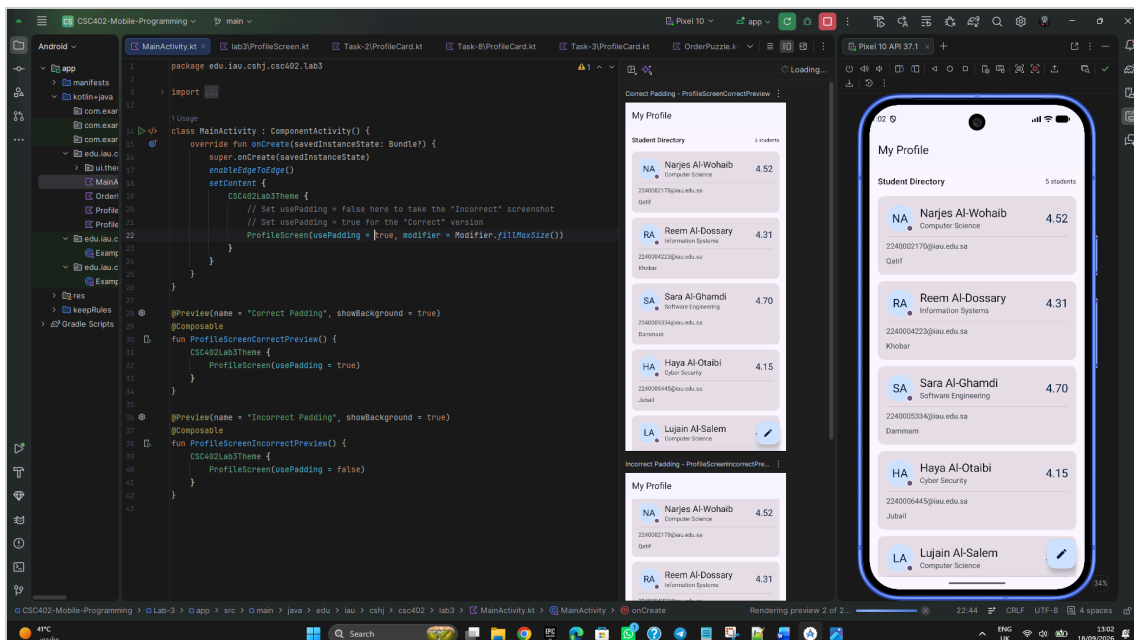


Light and Dark Previews Side-by-Side

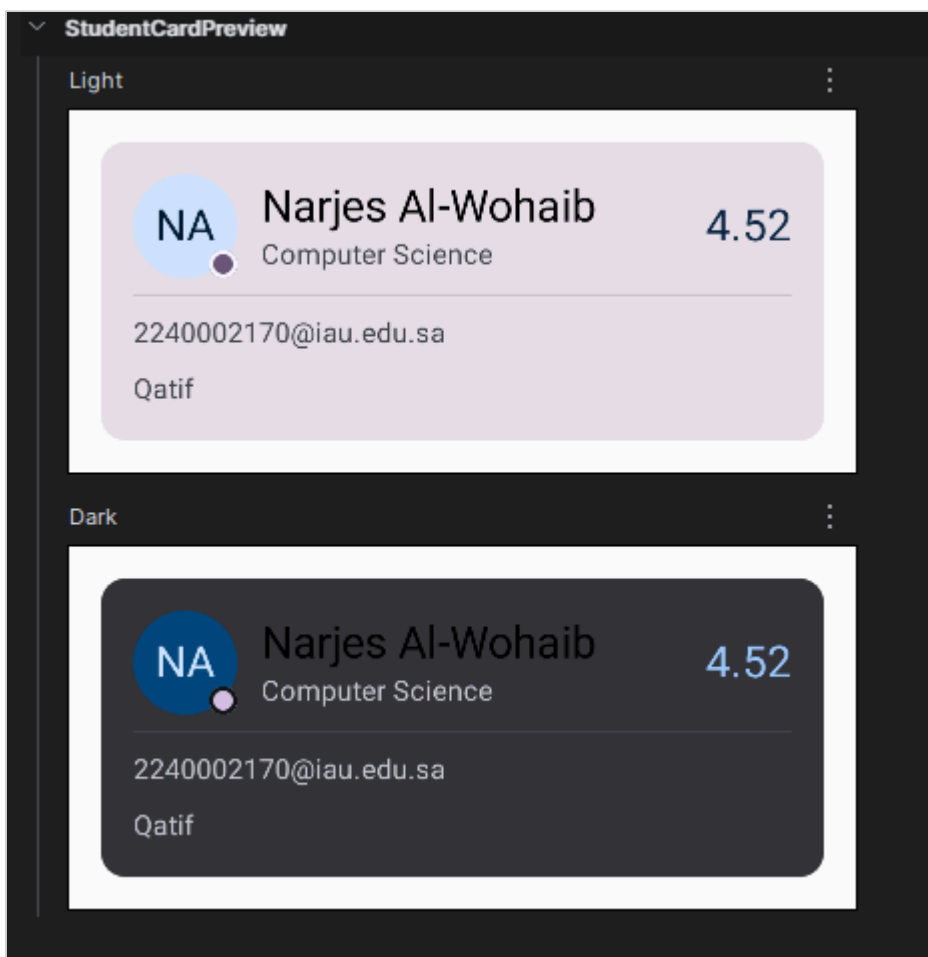


Scaffold without innerPadding





Q4 Dark Mode Verification (Temporary Test)



Q4 verification — temporary hard-coded Color.Black makes the student name difficult to read in dark mode.

4. AI Use & Disclosure Log

Date: 18 September 2026

Tools Used: ChatGPT, Android Studio Gemini

Nature of Use: AI tools were used during the completion of this lab and its documentation. ChatGPT and Android Studio Gemini provided guidance on understanding lab requirements, checking project file integrity, and assisting with general code/project cleanup. Additionally, they offered git/repository organization guidance, report formatting for the HTML write-up, and verification of semantic Material Theme 3 color role implementations across different task stages.

Verification: All responses and modifications were manually cross-checked with the functional project source code in the repository to ensure complete alignment with actual project behavior.